

© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2022

V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_13

13. Composite Pattern

Vaskaran Sarcar¹

(1) Garia, Kolkata, India

This chapter covers the Composite pattern.

GoF Definition

It composes objects into tree structures to represent part-whole hierarchies. The Composite pattern lets clients treat individual objects and compositions of objects uniformly.

Concept

Consider a shop that sells different kinds of dried fruit such as cashews, dates, and walnuts. Each item has a certain price. Let's assume that you can purchase individual items or you can purchase gift packs (or boxed items) which are composed of different dried fruit items. In this case, the cost of a gift pack is the sum of the individual parts (component). The composite pattern is useful in a similar situation where you want to treat both the individual parts and the combination of the parts in the same way so that you can process them uniformly.

In object-oriented programming, a composite is an object with a composition of one or more similar objects, where each of these objects has similar functionality. (This is also known as a “has-a” relationship among objects.) So, the usage of this pattern is common in tree-structured data. When you implement this pattern in such a data structure, you do not need to discriminate between a branch and the leaf nodes of the tree. You can achieve the following goals using this pattern:

- You can compose objects into a tree structure to represent a part-whole hierarchy.
- You can access both the composite objects (branches) and the individual objects (leaf nodes) uniformly. As a result, you can reduce the complexity of the code and make the application less error prone.

Real-Life Example

Apart from the previous example, you can also think of an organization that consists of many departments. In general, an organization has many employees. Some of these employees are grouped to form a department and those departments can be further grouped to build the final structure of the organization.

Computer World Example

This pattern is common in various UI frameworks where you need to add other UI controls such as a button or a textbox in a container. Here are some examples:

- I mentioned that a tree data structure can follow this concept where the clients can treat the leaves of the tree and the non-leaves (branches of the tree) in the same way. So, when you see hierarchical data, it's a clue that a composite pattern can be useful. XML files are very common examples where you notice such tree structures.
- In Java, note the use of the generic Abstract Window Toolkit (AWT) container object. It is a component that can contain other AWT components. For example, in the `java.awt.Container` class (which extends `java.awt.Component`) you can see overloaded versions of the `add(...)` method. These methods accept another `Component` object to proceed further.

Note When you traverse a tree, you often use the concept of the Iterator design pattern, covered in Chapter [18](#) of this book.

Implementation

This example represent a college organization. Let's assume you have a principal and two heads of departments (HODs), one for Computer Science and Engineering (CSE) and one for Mathematics (Math). In the Mathematics department, there are currently

two lecturers (or teachers), and in the Computer Science and Engineering department there are three lecturers (or teachers). The tree structures for this organization look like Figure **13-1**.

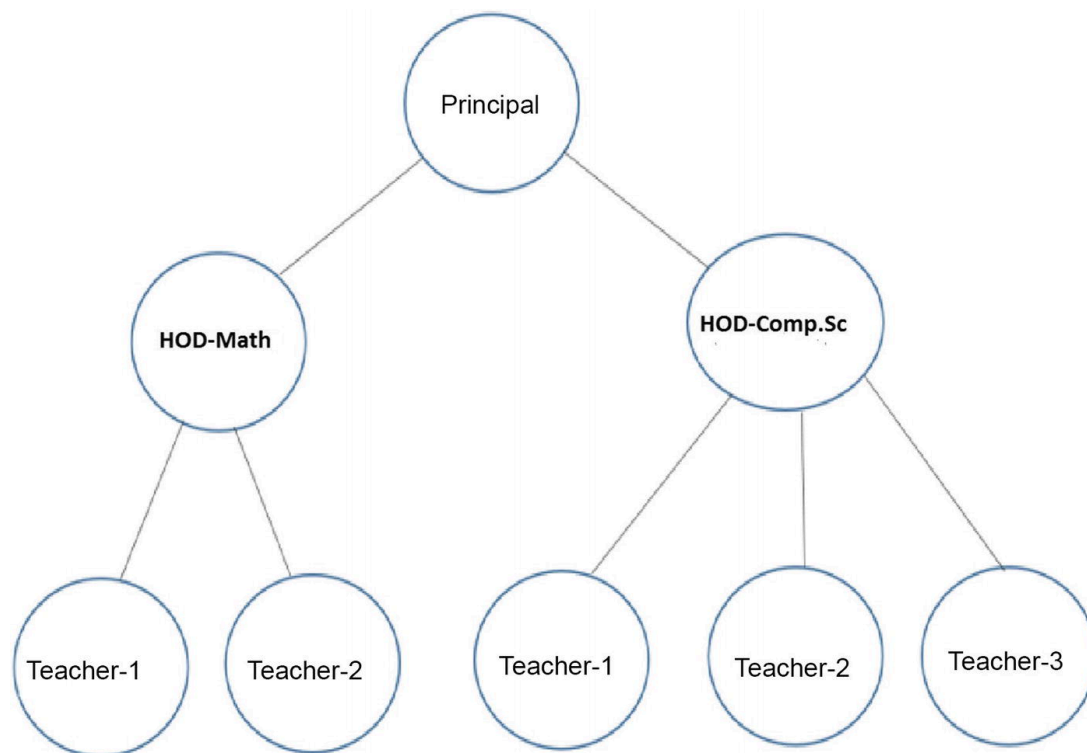


Figure 13-1 A college organization with a Principal, two HODs and five Lecturers

Let's also assume that at the end of the year, one lecturer from the CSE department resigns. The following example considers the mentioned scenarios.

Class Diagram

Figure **13-2** shows the class diagram.

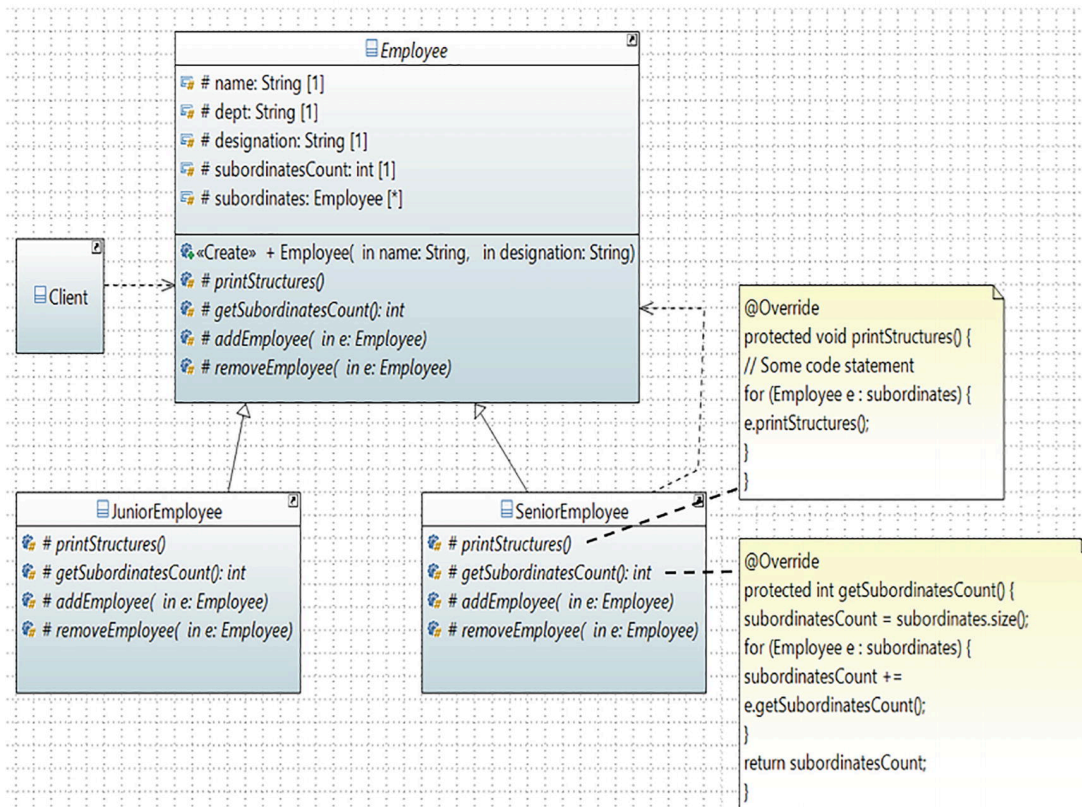


Figure 13-2 Class diagram

Package Explorer View

Figure **13-3** shows the high-level structure of the program.

- ▼  composite
 - >  Client.java
 - ▼  Employee.java
 - ▼  Employee
 - ◇ dept
 - ◇ designation
 - ◇ name
 - ◇ subordinates
 - ◇ subordinatesCount
 - ^c Employee(String, String)
 - ^A addEmployee(Employee) : void
 - ^A getSubordinatesCount() : int
 - ^A printStructures() : void
 - ^A removeEmployee(Employee) : void
 - ▼  JuniorEmployee.java
 - ▼  JuniorEmployee
 - ^c JuniorEmployee(String, String)
 - ◇ addEmployee(Employee) : void
 - ◇ getSubordinatesCount() : int
 - ◇ printStructures() : void
 - ◇ removeEmployee(Employee) : void
 - ▼  SeniorEmployee.java
 - ▼  SeniorEmployee
 - ^c SeniorEmployee(String, String)
 - ◇ addEmployee(Employee) : void
 - ◇ getSubordinatesCount() : int
 - ◇ printStructures() : void



Figure 13-3 Package Explorer view

Demonstration

This demonstration starts with the abstract class `Employee` that has four abstract methods. It looks like the following:

```
abstract class Employee {
    protected String name;
    protected String dept;
    protected String designation;
    protected int subordinatesCount;
    protected List<Employee> subordinates;
    // Constructor
    public Employee(String name, String designation) {
        this.name = name;
        this.designation=designation;
        this.subordinatesCount=0;
        subordinates = new ArrayList<Employee>();
    }
    protected abstract void printStructures();
    protected abstract int getSubordinatesCount();
    protected abstract void addEmployee(Employee e);
    protected abstract void removeEmployee(Employee e);
}
```

As per their names, it is easy to understand that the attributes `name`, `dept`, and `designation` are used to hold an employee name, corresponding department, and designation. The field `subordinatesCount` is used to show how many employees (or lecturers or teachers) directly report to another employee (typically a senior teacher).

The concrete classes `JuniorEmployee` and `SeniorEmployee` extend this `Employee` class. In the upcoming example, junior teachers do not supervise any other teacher so they act like leaf nodes of a tree. Conversely, one or more employees can report to a `HOD`, so it is treated as a non-leaf (or branch) node. Similarly, all `HODs` report to the `Principal`. This is why the

Principal is another non-leaf node. The HODs and the Principal are senior employees.

Here is the code segment to form a department. The comments are for your easy understanding.

```
System.out.println("Forming Mathematics department.");
System.out.println("A HOD with two lecturers are here.\n");
// Teachers of Mathematics directly
// reports to HOD-Mathematics
hodMaths.addEmployee(mathTeacher1);
hodMaths.addEmployee(mathTeacher2);
```

The `SeniorEmployee` class maintains the subordinate list. It has two additional methods called `addEmployee(...)` and `removeEmployee(...)`. These methods are used to add an employee to the list or remove an employee from the list, respectively.

Now go through the complete implementation and refer to the supportive comments. All parts of the program are stored inside the package `jdp3e.composite`.

```
// Employee.java
import java.util.ArrayList;
import java.util.List;

abstract class Employee {
    protected String name;
    protected String dept;
    protected String designation;
    protected int subordinatesCount;
    protected List<Employee> subordinates;
    // Constructor
    public Employee(String name, String designation) {
        this.name = name;
        this.designation=designation;
        this.subordinatesCount=0;
        subordinates = new ArrayList<Employee>();
    }
    protected abstract void printStructures();
    protected abstract int getSubordinatesCount();
    protected abstract void addEmployee(Employee e);
    protected abstract void removeEmployee(Employee e);
```

```
}  
  
// SeniorEmployee.java  
  
// It is a non-leaf node  
class SeniorEmployee extends Employee {  
    public SeniorEmployee(String name, String designation) {  
        super(name, designation);  
    }  
    @Override  
    protected void printStructures() {  
        System.out.println("\t" + this.name + " is a " +  
            this.designation);  
        for (Employee e : subordinates) {  
            e.printStructures();  
        }  
    }  
    @Override  
    protected int getSubordinatesCount() {  
        subordinatesCount = subordinates.size();  
        for (Employee e : subordinates) {  
            subordinatesCount += e.getSubordinatesCount();  
        }  
        return subordinatesCount;  
    }  
    @Override  
    protected void addEmployee(Employee e) {  
        subordinates.add(e);  
    }  
    @Override  
    protected void removeEmployee(Employee e) {  
        subordinates.remove(e);  
    }  
}  
  
// JuniorEmployee.java  
  
class JuniorEmployee extends Employee{  
    public JuniorEmployee(String name, String designation) {  
        super(name, designation);  
    }  
    @Override  
    protected void printStructures(){  
        System.out.println("\t\t"+this.name + " is a " +  
            this.designation);  
    }  
}
```



```

@Override
protected int getSubordinatesCount(){
    return subordinatesCount; //Always 0
}
@Override
protected void addEmployee(Employee e) {
    // Empty method
}
@Override
protected void removeEmployee(Employee e) {
    // Empty method
}
}
// Client.java
(Refer to the comments for your easy reference)
class Client {
    public static void main(String[] args) {
        System.out.println("***Composite Pattern
        Demonstration.***");
        Employee hodMath = formMathDept();
        Employee hodCompSc = formComputerScDept();
        Employee principal = formPrincipal(hodMath,
                                           hodCompSc);

        /*
        Printing the leaf-nodes and branches

        in the same way i.e.
        in each case, we are calling
        printStructures() method.
        */
        System.out.println("The details of the Principal object:");
        // Prints the complete college structure
        principal.printStructures();
        System.out.println("\n The details of the
                           HOD(Computer Sc.) object:");
        // Prints the details of Computer Science
        // department
        hodCompSc.printStructures();
        System.out.println("\n The details of the
                           HOD(Mathematics) object:");
        // Prints the details of Mathematics department
        hodMath.printStructures();
        // Leaf node

```

```

System.out.println("\n  The structure of a Junior
                        employee(leaf node):");
hodMath.subordinates.get(0).printStructures();
// Current management details
displayManagementDetail(principal,
                        hodCompSc,hodMath);
System.out.println("\nThe lecturer Mr.
                    C.Jones(cseTeacher2) resigns.");
hodCompSc.removeEmployee(
                        hodCompSc.subordinates.get(1));
System.out.println("The organization has the
                    following members now:");
principal.printStructures();
// Current management details
displayManagementDetail(principal,
                        hodCompSc,hodMath);
System.out.println("Testing the structure of some
                    junior employee's:");
Employee juniorMathTeacher1=
                        hodMath.subordinates.get(0);
Employee juniorCSETeacher1=
                        hodCompSc.subordinates.get(0);
System.out.println("The Lecturer( M.Jacob) manages
                    "+ juniorMathTeacher1.getSubordinatesCount()+
                    " employee(s).");
System.out.println("The Lecturer( C.Kate) manages
                    "+ juniorCSETeacher1.getSubordinatesCount()+
                    " employee(s).");
}
private static Employee formComputerScDept() {

    System.out.println("Forming Computer Science department.");
    System.out.println("A HOD with three lecturers is here.\n")
    Employee cseTeacher1 = new JuniorEmployee
                        ("C.Kate", "Lecturer");
    Employee cseTeacher2 = new JuniorEmployee
                        ("C.Jones", "Lecturer");
    Employee cseTeacher3 = new JuniorEmployee
                        ("C.Proctor", "Lecturer");
    Employee hodCompSc = new SeniorEmployee
                        ("Mr. V.Sarcar", "HOD(CSE)");
    // Teachers of Computer Science
    // directly reports to HOD-CSE

```

```

        hodCompSc.addEmployee(cseTeacher1);
        hodCompSc.addEmployee(cseTeacher2);
        hodCompSc.addEmployee(cseTeacher3);
        return hodCompSc;
    }

    private static Employee formMathDept() {
        System.out.println("Forming Mathematics department.");
        System.out.println("A HOD with two lecturers is here.\n");
        // Two lecturers other than HOD work
        // in the Mathematics department.
        Employee mathTeacher1 = new JuniorEmployee
            ("M.Jacob", "Lecturer");
        Employee mathTeacher2 = new JuniorEmployee
            ("M.Rustom", "Lecturer");
        Employee hodMath = new SeniorEmployee("Mrs.S.Das",
            "HOD(Math)");
        // Teachers of Mathematics directly
        // reports to HOD-Mathematics
        hodMath.addEmployee(mathTeacher1);
        hodMath.addEmployee(mathTeacher2);
        return hodMath;
    }

    private static Employee formPrincipal(Employee hodMath,
        Employee hodCompSc) {
        System.out.println("Forming the top-level management.");
        System.out.println("A principal with two HOD's is here.\n");
        // Principal of the college
        Employee principal = new
            SeniorEmployee("Dr.S.Som", "Principal");
        /*
        The Principal is on top of the college.
        The HOD(Mathematics) and
        HOD(Computer Science) directly report
        to him.
        */
        principal.addEmployee(hodMath);
        principal.addEmployee(hodCompSc);
        return principal;
    }

    private static void displayManagementDetail(
        Employee principal,
        Employee hodCompSc,
        Employee hodMaths) {
        System.out.println("\n---The current management

```

```

        status summary:---");
        System.out.println("The Principal manages "+
            principal.getSubordinatesCount()+ "
            employee(s).");
        System.out.println("The HOD(Computer Sc.) manages
            "+ hodCompSc.getSubordinatesCount()+ "
            employee(s).");
        System.out.println("The HOD(Mathematics) manages "+
            hodMaths.getSubordinatesCount()+ "
            employee(s).");
        System.out.println("-----");
    }
}

```

Output

Here's the output:

```

***Composite Pattern Demonstration.***
Forming Mathematics department.
A HOD with two lecturers is here.
Forming Computer Science department.
A HOD with three lecturers is here.
Forming the top-level management.
A principal with two HOD's is here.
The details of the Principal object:
    Dr.S.Som is a Principal
    Mrs.S.Das is a HOD(Math)
        M.Jacob is a Lecturer
        M.Rustom is a Lecturer
    Mr. V.Sarcar is a HOD(CSE)
        C.Kate is a Lecturer
        C.Jones is a Lecturer
        C.Proctor is a Lecturer
The details of the HOD(Computer Sc.) object:
    Mr. V.Sarcar is a HOD(CSE)
        C.Kate is a Lecturer
        C.Jones is a Lecturer
        C.Proctor is a Lecturer
The details of the HOD(Mathematics) object:
    Mrs.S.Das is a HOD(Math)
        M.Jacob is a Lecturer
        M.Rustom is a Lecturer

```

```

The structure of a Junior employee(leaf node):
    M.Jacob is a Lecturer
---The current management status summary:---
The Principal manages 7 employee(s).
The HOD(Computer Sc.) manages 3 employee(s).
The HOD(Mathematics) manages 2 employee(s).
-----
The lecturer Mr. C.Jones(cseTeacher2) resigns.
The organization has the following members now:
    Dr.S.Som is a Principal
    Mrs.S.Das is a HOD(Math)
        M.Jacob is a Lecturer
        M.Rustom is a Lecturer
    Mr. V.Sarcar is a HOD(CSE)
        C.Kate is a Lecturer
        C.Proctor is a Lecturer
---The current management status summary:---
The Principal manages 6 employee(s).
The HOD(Computer Sc.) manages 2 employee(s).
The HOD(Mathematics) manages 2 employee(s).
-----
Testing the structure of some junior employee's:
The Lecturer( M.Jacob) manages 0 employee(s).
The Lecturer( C.Kate) manages 0 employee(s).

```

Analysis

Before you move to the next section, let me remind you what I told you in the introduction of this book: *“Anything that is the latest today will be old (or outdated) tomorrow. But the core constructs (or features) are ever-green. All new features are built on top of these universal features. So, I like to write code that is compatible with a wide range of versions using the basic language constructs.”*

But you may have a different thought. For example, if you are a lambda lover, you may want to replace the code snippet

```

for (Employee e: subordinates) {
    e.printStructures();
}

```

with

```
subordinates.forEach(Employee :: printStructures);
```

You may refactor your code accordingly for similar segments. But lambda support comes in Java 8, not before that. If you ask me which one is better, my answer is that it's your choice. If my code is readable enough, easy to understand, can serve my purpose, and can run in a wide range of Java versions, I am OK with that. The same comment applies to all code in this book.

Q&A Session

13.1 What are the advantages of using the Composite design pattern?

Here are some advantages:

- In a tree-like structure, you can treat both the composite objects (branch nodes) and the individual objects (leaf nodes) uniformly. Notice that in this example, I use two common methods, `printStructures()` and `getSubordinatesCount()`, to print the structure of the organization and get the employee count from both the composite object structure (`Principal` or `HODs`) and the single object structure (leaf nodes like *M.Jacob*.)
- It is common to implement a part-whole hierarchy using this design pattern.
- You can easily add a new component to the architecture or delete an existing component from the architecture.

13.2 What are the challenges associated with using the Composite design pattern?

Here are some disadvantages:

- If you want to maintain the ordering of child nodes (for example, if the parse trees are represented as components), you may need to take special care.
- If you are dealing with immutable objects, you cannot simply delete them.
- If you keep adding new nodes, maintenance can become difficult over time. Sometimes you may want to deal with a composite that has spe-

cial components. This kind of constraint may cause additional development costs because you may need to provide a dynamic checking mechanism to support the concept.

13.3 In this example, you used a list data structure. Are other data structures OK to use?

Absolutely. There is no universal rule. You are free to use your preferred data structure. The GoF also confirmed that it is not necessary to use a general-purpose data structure.

13.4 How can you connect the Iterator design pattern to a Composite design pattern?

When you go through the example again, you will notice that if you want to examine a composite object architecture, you needed to iterate over the objects. This is why you see the following code segment in the `printStructure()` method:

```
for (Employee e: subordinates) {  
    e.printStructures();  
}
```

The `getSubordinatesCount()` method also uses a similar code snippet:

```
for (Employee e : subordinates) {  
    subordinatesCount += e.getSubordinatesCount();  
}
```

In addition, if you want to do special activities with some branches, you may need to iterate over the leaf nodes and non-leaf nodes.

13.5 In your implementation, the `addEmployee()` and `removeEmployee()` methods in the `JuniorEmployee` class are empty. You could use them for senior employees only. Is this understanding correct?

Nice observation. Even the GoF discussed this. It is true that when you put the `addEmployee(...)` and `removeEmployee(...)` methods in the abstract class, the leaf nodes (junior employees) have these addition and removal operations. Since a junior employee does not manage any other

employee, the presence of these methods is not meaningful enough for them. So, if you throw an exception instead of making the method body empty, you need to take special care when you use polymorphic code.

Notice that in this case, you need to update the client code as well. For example, now you cannot write

```
Employee hodMaths = new  
SeniorEmployee("Mrs.S.Das", "HOD(Math)");  
  
hodMaths.addEmployee(mathTeacher1);
```

But you can go ahead with

```
SeniorEmployee hodMaths = new  
SeniorEmployee("Mrs.S.Das", "HOD(Math)");  
  
hodMaths.addEmployee(mathTeacher1);
```

This is why, in this case, I opt for transparency over safety. Obviously, you promote more safety if you block the meaningless operations in the leaf nodes. This is why GoF also mentioned that this kind of decision involves a trade-off between safety and transparency.

13.6 I want to use an abstract class instead of an interface. Is this allowed?

In most cases, the simple answer is yes. But you need to understand the difference between an abstract class and an interface. In a typical scenario, you will find one of them may be more useful than the other one. Since throughout the book I present only simple and easy-to-understand examples, you may not see much difference between these two.

In Q&A Session [6.4](#) of Chapter [6](#), which covers the Builder pattern, I discuss how to decide between an abstract class and an interface.

Previous chapter

[**< 12. Flyweight Pattern**](#)

Next chapter

[**14. Bridge Pattern >**](#)